ICS 202 – Data Structures and Algorithms

Topic 14: Data Compression

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

Unless stated otherwise, Huffman trees are built as in the slides: the queue is in increasing order of frequency (ties in alphabetical order), the first tree dequeued becomes the left subtree, a merged tree is inserted after existing trees of equal priority, left edge = 0 and right edge = 1. Characters are 8-bit ASCII.

Q1. [new]

Which of the following is a *lossy* compression technique?

(A) Huffman coding

(B) LZ78

(C) JPEG

(D) Run-length encoding

Answer: (C). JPEG removes less important data based on perceptual models; the others are lossless.

My answer

Q2. [new]

Which statement about lossless and lossy compression is TRUE?

(A) Lossless compression removes perceptually less important data.

(B) Lossy techniques give higher compression rates, while lossless techniques are more accurate.

(C) In lossy compression the decompressed data is identical to the original.

(D) Lossless compression is used mainly for images, audio and video.

Answer: (B). Lossless only removes redundancy (exact recovery); lossy also drops data, so it compresses more.

My answer

Q3. [new]

Which of the following is TRUE about *static* Huffman coding?

(A) The mapping from symbols to codewords changes while the message is encoded.

(B) Only one scan of the message is needed.

(C) It needs two passes: one to compute the frequencies and build the code, and one to encode; the tree is sent with the bitstream.

(D) The sender does not transmit any model; the receiver builds it gradually.

Answer: (C). Static methods use a fixed mapping, two passes, and transmit the model (tree) before the bitstream.

My answer

Q4. [new]

The Huffman algorithm repeatedly dequeues the two trees with the smallest priorities and merges them. It is an example of:

(A) a divide-and-conquer algorithm

(B) a greedy algorithm

(C) a dynamic programming algorithm

(D) a backtracking algorithm

Answer: (B). At each step it makes the locally best choice (two smallest), so it is greedy.

My answer

Q5. [new]

Which statement about Huffman coding is TRUE?

(A) High-frequency symbols are assigned longer codewords.

(B) The Huffman code tree for a particular set of characters is not unique.

(C) All Huffman codewords of a message have the same length.

(D) A Huffman codeword may be a prefix of another Huffman codeword.

Answer: (B). Ties and the choice of left/right and 0/1 labels may be done differently, giving different trees.

My answer

Q6. Unit14 Activity 01

Construct the Huffman coding tree for the message `ONNOMONOMOO`. What is the codeword of each character?

(A) O = 0, N = 10, M = 11

(B) O = 1, N = 01, M = 00

(C) O = 1, N = 00, M = 01

(D) O = 00, N = 01, M = 1

Answer: (B). Frequencies O=6, N=3, M=2. Merge M(2)+N(3)=5 (M left); then 5 (left) + O(6) (right).

My answer

Q7.

Unit14 Activity 01

For the message `ONNOMONOMOO` coded with the Huffman code of the previous question, what are the number of bits BEFORE compression (8-bit ASCII), the number of bits of the compressed message (excluding the tree), and the compression ratio?

(A) 88, 16, 5.5

(B) 88, 46, 1.91

(C) 11, 16, 0.69

(D) 88, 22, 4.0

Answer: (A). Before: $11 \times 8 = 88$. After: $6 \cdot 1 + 3 \cdot 2 + 2 \cdot 2 = 16$. Ratio $= 88/16 = 5.5$.

My answer

Q8.

Unit14 Activity 01

A message contains (among others) the characters D, E and K with frequencies D = 42, E = 120, K = 7. Which of the following could be the Huffman codewords for K, D and E, respectively?

(A) 0, 101, 111101

(B) 101, 0, 111101

(C) 111101, 0, 101

(D) 111101, 101, 0

Answer: (D). The most frequent character (E) must get the shortest code and the least frequent (K) the longest.

My answer

Q9.

Unit14 Activity 01

Construct the Huffman coding tree for the characters C, D, E, K, L, M, U, Z with frequencies 32, 42, 120, 7, 42, 24, 37, 2. What are the codewords of E and Z?

(A) E = 0, Z = 111100

(B) E = 1, Z = 000011

(C) E = 0, Z = 11111

(D) E = 10, Z = 111101

Answer: (A). Codes: E=0, U=100, D=101, L=110, C=1110, M=11111, Z=111100, K=111101.

My answer

Q10.

Unit14 Activity 01

For the characters C, D, E, K, L, M, U, Z with frequencies 32, 42, 120, 7, 42, 24, 37, 2, how many bits are transmitted before compression (8-bit ASCII) and for the Huffman-encoded message (excluding the tree)?

(A) 2448 and 865

(B) 2448 and 785

(C) 306 and 785

(D) 2448 and 918

Answer: (B). $306 \times 8 = 2448$; $120 \cdot 1 + (37 + 42 + 42) \cdot 3 + 32 \cdot 4 + 24 \cdot 5 + (2 + 7) \cdot 6 = 785$.

My answer

Q11.

Unit14 Activity 01

For the same data (C, D, E, K, L, M, U, Z with frequencies 32, 42, 120, 7, 42, 24, 37, 2), what is the compression ratio when the Huffman tree is also transmitted (post-order scheme of the slides)?

(A) ≈ 3.12

(B) ≈ 0.35

(C) ≈ 2.83

(D) ≈ 2.45

Answer: (C). Tree: $8 \times 9 + 7 + 1 = 80$ bits; total $785 + 80 = 865$; ratio $= 2448/865 \approx 2.83$.

My answer

Q12.

Unit14 Activity 01

A Huffman coding tree has this shape (left edge = 0, right edge = 1): the root's left child is an internal node with leaves O (67, left) and T (85, right). The root's right child is an internal node N_1 whose right child is leaf E (120). N_1's left child is N_2 with right child leaf H (50). N_2's left child is N_3 with left child leaf G (17). N_3's right child is N_4 with right child leaf V (12). N_4's left child is N_5 with leaves X (4, left) and J (4, right). What is the bit sequence for "GXH"?

(A) 1000100100101

(B) 1000100101101

(C) 0111011011010

(D) 100010010101

Answer: (A). G = 1000, X = 100100, H = 101.

My answer

Q13.

Unit14 Activity 01

The string `BCAADDDCCACACAC` is to be transmitted; each character occupies 8 bits. Using Huffman encoding (and transmitting the Huffman tree as in the slides), how many bits are required without compression and with compression?

(A) 120 and 28

(B) 120 and 68

(C) 120 and 30

(D) 15 and 68

Answer: (B). Codes C=0, B=100, D=101, A=11: message $6+3+9+10 = 28$ bits; tree $4\times9+3+1 = 40$; total 68.

My answer

Q14.

Unit14 Activity 01

Using the Huffman code for `BCAADDDCCACACAC` (C = 0, B = 100, D = 101, A = 11), decode `11100`.

(A) BA

(B) AB

(C) ADC

(D) AAC

Answer: (B). 11 = A, 100 = B.

My answer

Q15.

Unit14 Activity 01

Using the same code (C = 0, B = 100, D = 101, A = 11), decode `1001`.

(A) B

(B) BA

(C) DC

(D) It cannot be decoded

Answer: (D). 100 = B, then the last bit 1 leads to an internal node, not a leaf.

My answer

Q16.

Unit14 Activity 01

Construct the Huffman coding tree for {A, B, C, D, E} with frequencies {0.17, 0.11, 0.24, 0.33, 0.15}. Then decode the bit sequence `1000001101`.

(A) BACE

(B) EACB

(C) BADE

(D) DACE

Answer: (A). Codes A=00, C=01, B=100, E=101, D=11: 100—00—01—101.

My answer

Q17.

Unit14 Activity 01

Which of the following sets of codewords satisfies the prefix-free property?

(A) {0, 00, 100, 111}

(B) {0, 100, 101, 11}

(C) {0, 1, 01, 11}

(D) {110, 101, 11, 10}

Answer: (B). In the others, 0 is a prefix of 00; 0 of 01 and 1 of 11; 11 of 110.

My answer

Q18.

Unit14 Activity 01

Which of the following sets of codewords does NOT satisfy the prefix-free property?

(A) {1, 01, 000, 001}

(B) {0, 100, 101, 11}

(C) {110, 101, 11, 10}

(D) {00, 01, 10, 11}

Answer: (C). 11 is a prefix of 110 (and 10 of 101).

My answer

Q19. Unit14 Activity 01

Consider three encodings: E1: A=0, B=1, C=11, D=10; E2: K=0, L=1, M=11, N=10; E3: S=0, T=101, A=11, B=100. Which are bad encodings (not uniquely decodable because the prefix property fails)?

(A) E1 only

(B) E3 only

(C) E1 and E2 only

(D) E1, E2 and E3

Answer: (C). In E1 and E2, 1 is a prefix of 11 and 10; E3 is prefix-free.

My answer

Q20. Unit14 Activity 01

A Huffman tree has this shape: the root's left child is an internal node with leaves O (left) and E (right); the root's right child is an internal node whose left child is an internal node with leaves A (left) and P (right), and whose right child is leaf H. What is the bitstring representation of the tree (post-order, as in the slides)?

(A) 1O1E01A1P01H00

(B) 1O1E01A1P01H000

(C) 01O1E01A1P1H00

(D) 1H1P1A01E1O000

Answer: (B). Post-order: 1O 1E 0, 1A 1P 0, 1H 0, root 0, then the end-of-tree 0.

My answer

Q21. Unit14 Activity 01

For the tree of the previous question (5 leaves O, E, A, P, H), the number of bits required to transmit the tree (each character in 8-bit ASCII) is:

(A) $25 <$ bits < 35

(B) $35 <$ bits < 45

(C) $45 <$ bits < 55

(D) $55 <$ bits

Answer: (C). $5 \times (1 + 8) + 4$ internal 0-bits $+1$ end bit $= 50$.

My answer

Q22. Unit14 Activity 01

Reconstruct the Huffman tree from the encoded string `1C1E1D001B1A000` (left edge = 0, right edge = 1). What is the codeword of D?

(A) 010

(B) 011

(C) 10

(D) 001

Answer: (B). The tree is ((C, (E, D)), (B, A)): C=00, E=010, D=011, B=10, A=11.

My answer

Q23. [new]

Using the Huffman codewords a = 110, e = 10, l = 0110, n = 111, o = 0111, s = 010, t = 00, the encoding of the message `test` is:

(A) 0010010000

(B) 001001000

(C) 001000100

(D) 0001001000

Answer: (B). 00—10—010—00.

My answer

Q24. [new]

Using the Huffman tree with codewords a = 110, e = 10, l = 0110, n = 111, o = 0111, s = 010, t = 00, decode `10100010111010001000010011`.

(A) eetenstetsl

(B) eetenstetso

(C) Decoding fails at the first bit.

(D) After "eetenstets" decoding fails: the remaining bits 011 do not reach a leaf.

Answer: (D). 10—10—00—10—111—010—00—10—00—010 then 011 ends at an internal node.

My answer

Q25. [new]

A Huffman tree with 12 leaves (12 distinct characters) is stored using the post-order scheme of the slides (1-bit + 8-bit ASCII per leaf, 0-bit per internal node, one extra 0-bit at the end). How many bits are needed?

(A) 108

(B) 119

(C) 120

(D) 96

Answer: (C). $12 \times 9 + 11 + 1 = 120$.

My answer

Q26. [new]

When recovering a transmitted Huffman tree with a stack, a 0-bit is read and the stack contains more than one element. What happens?

(A) The next byte is read and pushed onto the stack.

(B) The top two elements are popped; the first popped becomes the left child, the second the right child; the new node is pushed.

(C) The top two elements are popped; the first popped becomes the right child, the second the left child; the new node is pushed.

(D) The tree construction is complete.

Answer: (C). As in the slides: 0 means pop two, first popped = right child, second = left child, push the parent.

My answer

Q27. [new]

With the codewords a = 0, b = 101, c = 100, d = 111, e = 110, f = 1100, the bitstring `11000100110` can be decoded both as “face” and as “eaace”. Why?

(A) Because a = 0 is too short.

(B) Because e’s codeword 110 is a prefix of f’s codeword 1100.

(C) Because b and c have the same length.

(D) Because the bitstring length is odd.

Answer: (B). The prefix property is violated, so the code is not uniquely decodable.

My answer

Q28. [new]

A message contains the characters A, B, C, D, E with frequencies 5, 2, 2, 1, 1. How many bits does the Huffman-encoded message need (excluding the tree)?

(A) 21

(B) 23

(C) 24

(D) 33

Answer: (B). Codes A=0, D=100, E=101, B=110, C=111: $5 \cdot 1 + (2+2+1+1) \cdot 3 = 23$.

My answer

Q29. [new]

A message of 261 characters (a, e, l, n, o, s, t with frequencies 45, 65, 13, 45, 18, 22, 53) is Huffman-encoded into 696 bits, and the Huffman tree is transmitted with it using 70 bits. Compared with 8-bit ASCII, the compression ratio is approximately:

(A) 3.00

(B) 2.72

(C) 0.37

(D) 2.38

Answer: (B). $2088/(696+70) = 2088/766 \approx 2.72$ (3.00 ignores the tree).

My answer

Q30. Unit14 Activity 02

Which pair is produced in LZ78 encoding?

(A) (Character, Frequency)

(B) (Offset, Length)

(C) (Index, Character)

(D) (Bit, Byte)

Answer: (C). Each output is (dictionary index of the prefix, next character).

My answer

Q31. [new]

In LZ78, the dictionary of patterns is usually implemented as:

(A) a stack

(B) a hash table

(C) a binary heap

(D) a circular queue

Answer: (B). Stated in the slides; fast lookup of patterns.

My answer

Q32.

Unit14 Activity 02

Compress the string `ABBCBCABABCAABCAAB` using LZ78. The result is:

(A) (0,A)(0,B)(2,C)(3,A)(2,A)(4,A)(6,B)

(B) (0,A)(0,B)(2,C)(3,A)(2,A)(4,A)(6,B)(0,D)

(C) (0,A)(0,B)(1,C)(3,A)(1,A)(4,A)(6,B)

(D) (0,A)(0,B)(2,C)(2,C)(2,A)(4,A)(6,B)

Answer: (A). Patterns A, B, BC, BCA, BA, BCAA, BCAAB.

My answer

Q33.

[new]

Encode the string `ABABABA` using LZ78.

(A) (0,A)(0,B)(1,B)(3,A)

(B) (0,A)(0,B)(1,B)(1,B)(1,)

(C) (0,A)(0,B)(1,B)(3,)

(D) (0,A)(0,B)(2,A)(3,A)

Answer: (A). Patterns A, B, AB, ABA.

My answer

Q34.

[new]

Encode the string `SATATASACITASA` using LZ78.

(A) (0,S)(0,A)(0,T)(2,T)(2,S)(2,C)(0,I)(3,A)(1,A)

(B) (0,S)(0,A)(0,T)(2,T)(4,S)(2,C)(0,I)(3,A)(1,A)

(C) (0,S)(0,A)(0,T)(2,T)(2,S)(2,C)(0,I)(3,A)(1,)

(D) (0,S)(0,A)(0,T)(1,T)(1,S)(1,C)(0,I)(2,A)(0,A)

Answer: (A). Patterns S, A, T, AT, AS, AC, I, TA, SA.

My answer

Q35. Unit14 Activity 02

Decode the LZ78 encoded sequence (0,A) (0,B) (2,C) (3,A).

(A) ABCA

(B) ABBCBCA

(C) ABBCA

(D) ABBCBCAB

Answer: (B). A, B, BC, BCA.

My answer

Q36. [new]

Decode the LZ78 encoded sequence (0,X) (0,Y) (1,Y) (2,X) (2,Y) (1,).

(A) XYXYYXYYX

(B) XYXYXYYX

(C) XYYXYXYYX

(D) XYXYYXYY

Answer: (A). X, Y, XY, YX, YY, X.

My answer

Q37. [new]

In LZ78, codewords are indexed starting from 1. How many bits are used for the integer part of the codeword with index 9?

(A) 3

(B) 4

(C) 8

(D) 9

Answer: (B). $\lceil \log_2 9 \rceil = 4$ (equivalently, $i - 1 = 8 = 1000_2$ has 4 significant bits).

My answer

Q38. [new]

The LZ78 encoding of `SATATASACITASA` is (0,S)(0,A)(0,T)(2,T)(2,S)(2,C)(0,I)(3,A)(1,A). Using 8 bits per character and the slides' rule for the integer part, how many bits are transmitted?

(A) 112

(B) 90

(C) 94

(D) 99

Answer: (C). $(1+8)+(1+8)+(2+8)+(2+8)+4(3+8)+(4+8)=94$.

My answer

Q39.

Unit14 Activity 02

Why does LZ78 not need to transmit the dictionary separately?

(A) Because the dictionary is fixed and known in advance for all messages.

(B) Because the decoder rebuilds the same dictionary from the (index, character) pairs as it decodes.

(C) Because the dictionary is stored in the first byte of the message.

(D) Because LZ78 is a lossy method and does not need it.

Answer: (B). Each decoded pair adds exactly the entry the encoder added, so both sides build the dictionary gradually.

My answer

Q40.

Unit14 Activity 02

Can two different messages produce the same LZ78 encoding?

(A) Yes, whenever they have the same characters.

(B) Yes, if both messages have the same length.

(C) No, because decoding is deterministic: an encoding decodes to exactly one message.

(D) No, because LZ78 sends the dictionary with the message.

Answer: (C). LZ78 is lossless; the pairs determine the dictionary and the output uniquely.

My answer

Part B: Programming

Use the following classes where needed:

```
class HuffmanNode {
    char symbol; int freq; HuffmanNode left, right;
    HuffmanNode(char symbol, int freq) { this.symbol = symbol; this.freq = freq; }
    HuffmanNode(HuffmanNode left, HuffmanNode right) {
        this.left = left; this.right = right; this.freq = left.freq + right.freq;
    }
    boolean isLeaf() { return left == null && right == null; }
}
class LZ78Pair {
    int index; Character ch;          // ch == null for a final (index, ) pair
    LZ78Pair(int index, Character ch) { this.index = index; this.ch = ch; }
    public String toString() { return "(" + index + "," + (ch == null ? "" : ch) + ")"; }
}
```

Q41. [new]

Write `public static HuffmanNode buildHuffmanTree(char[] chars, int[] freq)` that builds the Huffman tree with the slides' algorithm: put one-node trees in a queue in increasing order of frequency (ties alphabetically); while more than one tree remains, dequeue t_1 and t_2, create t with t_1 as left and t_2 as right subtree, and insert t in its proper location (after trees of equal priority).

Answer:

```
public static HuffmanNode buildHuffmanTree(char[] chars, int[] freq) {
    List<HuffmanNode> queue = new ArrayList<>();
    for (int i = 0; i < chars.length; i++) queue.add(new HuffmanNode(chars[i], freq[i]));
    queue.sort((a, b) -> a.freq != b.freq ? a.freq - b.freq : a.symbol - b.symbol);
    while (queue.size() > 1) {
        HuffmanNode t1 = queue.remove(0);
        HuffmanNode t2 = queue.remove(0);
        HuffmanNode t = new HuffmanNode(t1, t2);
        int pos = 0;
        while (pos < queue.size() && queue.get(pos).freq <= t.freq) pos++;
        queue.add(pos, t);
    }
    return queue.get(0);
}
```

My answer

Q42. [new]

Write `public static void generateCodes(HuffmanNode node, String prefix, Map<Character, String> table)` that stores in `table` the codeword of every leaf (path from the root, left = "0", right = "1"). It is first called as `generateCodes(root, "", table)`.

Answer:

```
public static void generateCodes(HuffmanNode node, String prefix, Map<Character, String>
    table) {
    if (node == null) return;
    if (node.isLeaf()) { table.put(node.symbol, prefix); return; }
    generateCodes(node.left, prefix + "0", table);
    generateCodes(node.right, prefix + "1", table);
}
```

My answer

Q43. [new]

Write `public static String encode(String message, Map<Character, String> table)` that replaces every symbol of the message with its Huffman codeword. Return `null` if some symbol has no codeword.

Answer:

```
public static String encode(String message, Map<Character, String> table) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < message.length(); i++) {
        String code = table.get(message.charAt(i));
        if (code == null) return null;
        sb.append(code);
    }
    return sb.toString();
}
```

My answer

Q44. [new]

Write `public static String decode(String bits, HuffmanNode root)` that decodes a bitstream: start at the root, go left on 0 and right on 1; when a leaf is reached output its character and restart at the root. Return `null` if the bitstream cannot be decoded (it ends before reaching a leaf).

Answer:

```
public static String decode(String bits, HuffmanNode root) {
    StringBuilder out = new StringBuilder();
    HuffmanNode p = root;
    for (int i = 0; i < bits.length(); i++) {
        p = (bits.charAt(i) == '0') ? p.left : p.right;
        if (p == null) return null;
        if (p.isLeaf()) { out.append(p.symbol); p = root; }
    }
    return (p == root) ? out.toString() : null;
}
```

My answer

Q45. [new]

Write `public static boolean isPrefixFree(String[] codes)` that returns `true` if no codeword in the array is a prefix of another codeword (the prefix property). Example: {0, 100, 101, 11} gives `true`; {110, 101, 11, 10} gives `false`.

Answer:

```
public static boolean isPrefixFree(String[] codes) {
    for (int i = 0; i < codes.length; i++)
```

```
        for (int j = 0; j < codes.length; j++)
            if (i != j && codes[j].startsWith(codes[i])) return false;
    return true;
}
```

My answer

Q46. [new]

Write `public static String encodeTree(HuffmanNode root)` that returns the representation used to transmit a Huffman tree: a post-order traversal writing "1" followed by the character for each leaf and "0" for each internal node, then one more "0" to mark the end. (For the slides' tree the result is `1t1s1l1o0001e1a1n0000`.)

Answer:

```
public static String encodeTree(HuffmanNode root) {
    StringBuilder sb = new StringBuilder();
    postOrderWrite(root, sb);
    sb.append('0');                       // end of tree
    return sb.toString();
}
private static void postOrderWrite(HuffmanNode node, StringBuilder sb) {
    if (node == null) return;
    postOrderWrite(node.left, sb);
    postOrderWrite(node.right, sb);
    if (node.isLeaf()) sb.append('1').append(node.symbol);
    else sb.append('0');
}
```

My answer

Q47. [new]

Write `public static HuffmanNode recoverTree(String s)` that rebuilds a Huffman tree from a string such as `1C1E1D001B1A000` using a stack: on "1" push a leaf for the next character; on "0", if the stack has one element the tree is complete, otherwise pop two nodes (first popped = right child, second = left child) and push their new parent.

Answer:

```
public static HuffmanNode recoverTree(String s) {
    Stack<HuffmanNode> stack = new Stack<>();
    int i = 0;
    while (i < s.length()) {
        if (s.charAt(i) == '1') {
            stack.push(new HuffmanNode(s.charAt(i + 1), 0));
            i += 2;
        } else {
            if (stack.size() == 1) return stack.pop();
            HuffmanNode right = stack.pop();
            HuffmanNode left = stack.pop();
            stack.push(new HuffmanNode(left, right));
            i++;
        }
    }
    return null; // malformed string
}
```

My answer

Q48. [new]

Given a Huffman tree whose leaves store the character frequencies (the root's `freq` is the message length), write `public static double compressionRatio(HuffmanNode root)` that returns (bits before compression) / (bits after compression), where before = 8 bits per character and after = encoded message bits + bits to transmit the tree (9 per leaf, 1 per internal node, plus 1 end bit). For the slides' example it returns $2088/766$.

Answer:

```
public static int messageBits(HuffmanNode node, int depth) {
    if (node == null) return 0;
    if (node.isLeaf()) return node.freq * depth;
    return messageBits(node.left, depth + 1) + messageBits(node.right, depth + 1);
}
public static int treeBits(HuffmanNode node) {
    if (node == null) return 0;
```

```
    if (node.isLeaf()) return 9;
    return 1 + treeBits(node.left) + treeBits(node.right);
}
public static double compressionRatio(HuffmanNode root) {
    int before = 8 * root.freq;
    int after = messageBits(root, 0) + treeBits(root) + 1;
    return (double) before / after;
}
```

My answer

Q49. [new]

Write `public static List<LZ78Pair> lz78Encode(String s)` that encodes a string with LZ78 using a `HashMap<String, Integer>` dictionary (index 0 = empty string). If the input ends while the current prefix is in the dictionary, output `(index, )` with a `null` character. Example: `BABAABRRRA` gives (0,B)(0,A)(1,A)(2,B)(0,R)(5,R)(2,).

Answer:

```
public static List<LZ78Pair> lz78Encode(String s) {
    Map<String, Integer> dict = new HashMap<>();
    List<LZ78Pair> out = new ArrayList<>();
    String p = "";
    for (int i = 0; i < s.length(); i++) {
        char k = s.charAt(i);
        String pk = p + k;
        if (dict.containsKey(pk)) p = pk;          // extend the prefix
        else {
            int w = p.isEmpty() ? 0 : dict.get(p);
            out.add(new LZ78Pair(w, k));
            dict.put(pk, dict.size() + 1);
            p = "";
        }
    }
    if (!p.isEmpty()) out.add(new LZ78Pair(dict.get(p), null));
    return out;
}
```

My answer

Q50. [new]

Write `public static String lz78Decode(List<LZ78Pair> pairs)` that decodes LZ78 pairs: for index 0 output the character, otherwise output the dictionary string at that index followed by the character; insert each output into the dictionary (a final pair with a `null` character only outputs the dictionary string).

Answer:

```
public static String lz78Decode(List<LZ78Pair> pairs) {
    List<String> dict = new ArrayList<>();
    dict.add("");                           // index 0 = empty string
    StringBuilder out = new StringBuilder();
    for (LZ78Pair pr : pairs) {
        String entry = dict.get(pr.index) + (pr.ch == null ? "" : pr.ch);
        out.append(entry);
        if (pr.ch != null) dict.add(entry);
    }
    return out.toString();
}
```

My answer